0计算机基础的Agent应用与开发入门 · Lesson 3

Lesson 3: How Software Connects - A Search from Your Phone to a Remote Service and Back

distinguish local and remote parts of an application; explain a search using clients, servers, requests, and responses; describe what an API specifies; draw data flowing between devices; and distinguish a failed connection, a service error, and a successful search with no matches.

Author: Ritten_poem

> **In this lesson you will learn to:** distinguish local and remote parts of an application; explain a search using clients, servers, requests, and responses; describe what an API specifies; draw data flowing between devices; and distinguish a failed connection, a service error, and a successful search with no matches.

1. Pick up where the last lesson ended

In Lesson 2 you searched a map for a nearby convenience store. We divided the work into locating you, reading business records, filtering, sorting, and displaying results. We also labeled the arrows with the information they carried.

Now ask another question: **are all those modules and records on your phone?**

Installing a map app does not necessarily put every business's latest details on your device. Shops open and close; opening hours change. You may use the same account on another device. How would separate copies on every phone stay up to date?

One common design keeps the interaction on the phone and asks a program running elsewhere to perform the search. The result travels back. You see one list, but software on different devices may have worked together to produce it.

We will build a **simplified map system for teaching**. Its architecture, businesses, and numbers are illustrative; this is not a description of any particular map product. Real products can divide the work differently and may use offline records or cached data.

2. Ask where the work runs

A module's responsibility answers “what does this part do?” Local and remote answer “where does this part run?” They are different questions.

Local is relative to the device you are using. Here it means the phone. **Remote** means another device contacted through a network. That device could be in the next room; it need not be geographically far away.

For our example, we will divide the work like this:

Location	Work performed	Data used
Local phone	Collect search terms and location, send the query, display returned results	Search terms, location, returned business records
Remote program	Check query conditions, read records, filter, calculate distances, sort, and prepare results	Business records, query conditions, calculated results

Filtering and sorting could also happen on the phone. We put them remotely here to make the whole path easy to follow. **Responsibility and placement are separate design choices.**

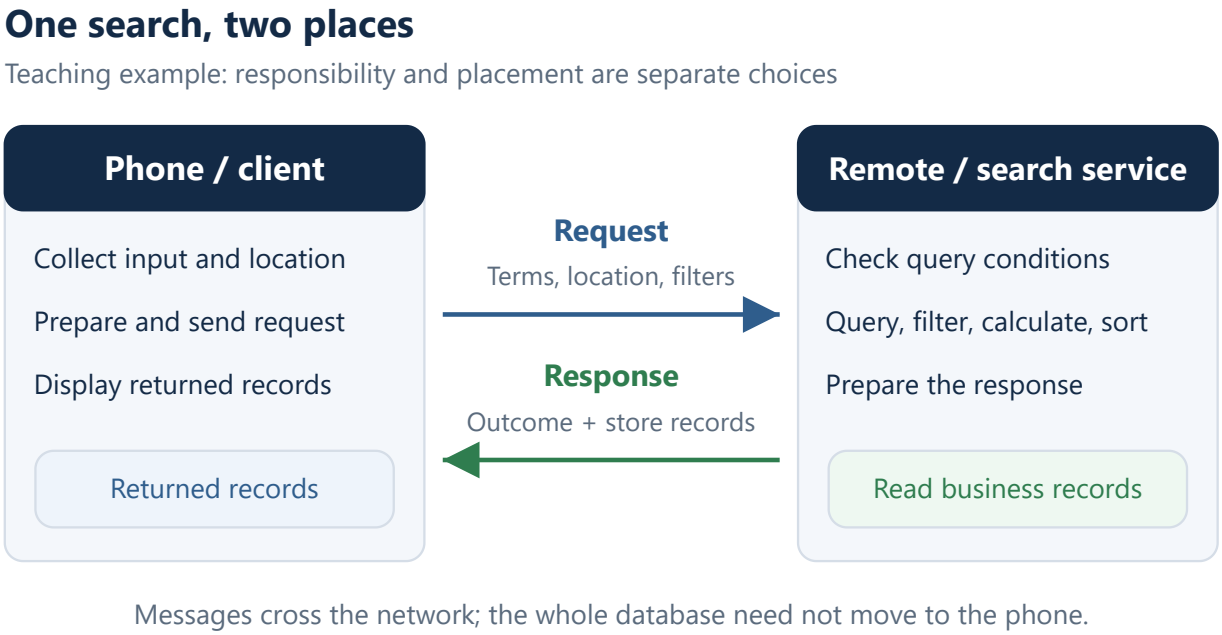


Figure 3-1. Query conditions cross the network in one direction; result data comes back. The phone does not need the whole business database for this search.

Read along the diagram: you give conditions to the phone; the phone sends them remotely; the remote program uses business records to prepare a result; the phone turns that result into a list. The response might contain two business records rather than a finished screen image.

> **A tip:** Lesson 1 focused on data that must be saved. Search terms, messages in transit, and temporary calculation results are data too, even when they are not kept long term. Transmitting information does not by itself mean it has been saved.

3. Networks connect devices

Imagine one person greeting customers and another looking up records. If they work in different rooms, they need a way to communicate. Programs on different devices need one too.

A **computer network** connects devices so programs on them can exchange data. Your phone might join through Wi-Fi or a mobile connection, then reach its destination through several network devices. For now, remember two things: the data needs a path, and the programs need rules they both understand.

A network is not necessarily the internet. Devices in a home can form a local network; the internet interconnects many networks. Our focus is how applications use a network to cooperate.

1. Clients and servers are roles

A **client** is a program that initiates a request to use a service. The phone's map program is the client in this exchange.

A **server** is a program that receives requests, handles them, and returns responses. Here it is the remote business-search program. The computer hosting it is also often called a server; distinguish that machine from the program running on it.

A **service** is a capability a program makes available for others to use, such as "find nearby convenience stores." You can use it without keeping every business record yourself or knowing how its internal modules work.

These are roles, not labels for device sizes. A browser on a powerful desktop can be a client. A server requesting information from another service acts as a client in that exchange. Client and server programs can also run on the same computer.

2. Requests and responses carry the exchange

A **request** is a message from the client describing what it wants done and providing the information needed to do it.

A **response** is a message returned by the server with the result or an explanation of why it could not complete the request.

"Find something nearby" is not enough. Nearby where? What kind of business? How large an area? The remote program needs those conditions.

Direction	Information sent	Teaching example
Phone to remote program: request	Operation and query conditions	Find convenience stores; location A; within 1,000 meters; open now
Remote program to phone: response	Outcome and business records	Search succeeded; two stores: Sunrise at 300 meters, Corner at 800 meters

This table describes messages in ordinary language. **It is not a real API message you can send.** Location A stands in for fictional coordinates so you can focus on what the information does.

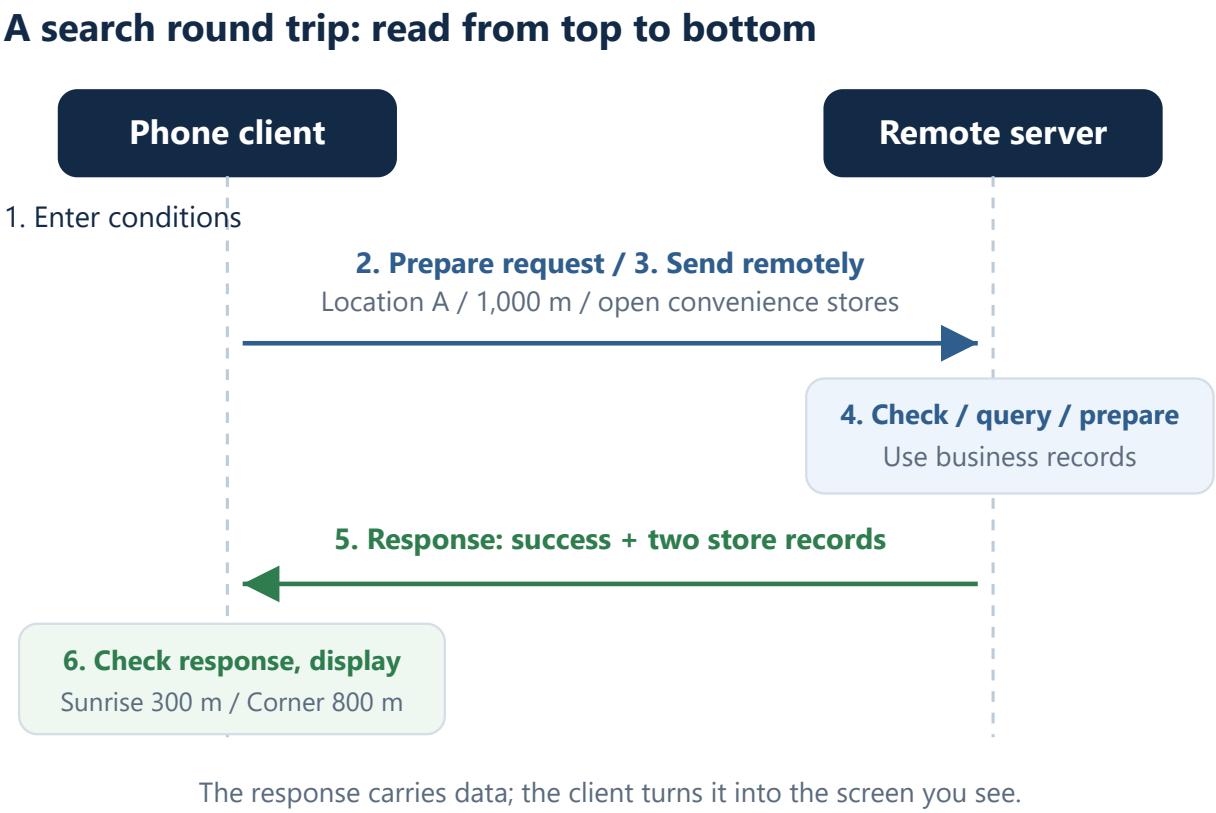


Figure 3-2. Time runs from top to bottom. The request goes out and the response returns; querying records and displaying results happen in different places.

Follow six steps: 1) you provide conditions; 2) the phone prepares a request; 3) the request travels to the remote program; 4) it checks the conditions, searches, and prepares results; 5) the response travels back; 6) the phone checks the response and displays results or a message.

Connect this to Lesson 1: input goes through processing performed jointly by the phone and remote program, then becomes output. The input-processing-output structure is still there. The work is simply spread across devices.

4. Find the other side and agree on the exchange

1. IP, domain names, and DNS: who do we contact?

An **IP address** is used for addressing on a network. A **domain name** is a name that is easier for people to use. **DNS** helps resolve domain names into the address information needed to contact a destination. An address book is a useful first analogy, though a real lookup need not be a single step.

A domain can have multiple IP addresses, and addresses can change. You do not need to memorize their forms. The point is that a name must lead to the information needed to reach the destination.

A **URL** also specifies how to access something and which resource or entry point is wanted. In the illustrative address `https://maps.example/search`, `https` specifies HTTPS, `maps.example` is the domain, and `/search` is the path. This is a teaching address, not a working map service.

2. HTTP(S): what communication rules do we use?

Finding an address is not enough. The programs must also agree how messages and outcomes are expressed. **HTTP** is an application-layer protocol used by websites and many network APIs. It provides the request-and-response pattern we use here. Other network communication can use other protocols.

HTTPS uses HTTP with encrypted protection. It helps protect information in transit and authenticate the service you connect to. It does not guarantee accurate shop opening hours or correct application logic.

3. APIs: what can we ask the other program to do?

An **API**, or application programming interface, is an interface for programs to use. For this lesson, think of it as an actionable agreement: available operations, required inputs, expected outputs, and ways to express failure.

Our business-search service offers a network API for finding nearby stores. The client supplies conditions in the agreed form; the server returns results in the agreed form. The API is the entry point and contract; the service is the capability behind it.

Question the API must settle	Our teaching example
What operation is available?	Find nearby convenience stores that are open
What inputs are required?	Location, business category, search radius; the open-only filter is optional
What input limits apply?	A positive radius; this example accepts 100 to 5,000 meters
What does success return?	A success indicator and a list with each store's name, distance, and opening status
What if there are no matches?	Success with an empty store list
What if input is invalid?	An explicit error, rather than pretending there were no stores

The radius limit is a rule we invented for this example, not an industry standard. Real APIs may also specify authentication, usage limits, or charges. Our exercises use fictional data and require no account or API key.

> **A common mistake:** an API is not the same as a network or a web page. Programs also use APIs to access capabilities on their own computer. We are studying network service calls here; the existence of an API alone does not prove a program goes online.

Lesson 2 described responsibilities inside software. An API describes how a caller can use a capability exposed by another part. The caller usually does not need to understand that part's entire implementation.

5. “No results” can mean different things

An empty screen does not prove the server is broken. Ask: was a request sent? Did a response arrive? Did it report success or failure? If successful, did it contain records?

What you observe	What it tells you	What to check next
The phone reports no network and did not send the request	The exchange has not completed	Device connectivity and the app's message
A request was sent, but no result arrived before the deadline	A timeout occurred; the outcome is uncertain	Connectivity, service records, and whether a result already exists
A response says access is forbidden	A reply arrived, but access was refused	Account, credentials, and permissions
A response reports an internal service error	The service reported a processing problem	Service status and error details
A successful response contains an empty list	The search completed with no matches under these conditions	Location, radius, and filters
Two records arrived, but the screen shows none	Data arrived; later handling or display may be wrong	Response contents, client processing, and display logic

These are directions for investigation, not proof of a unique cause. A connected network icon does not guarantee that a particular service is reachable.

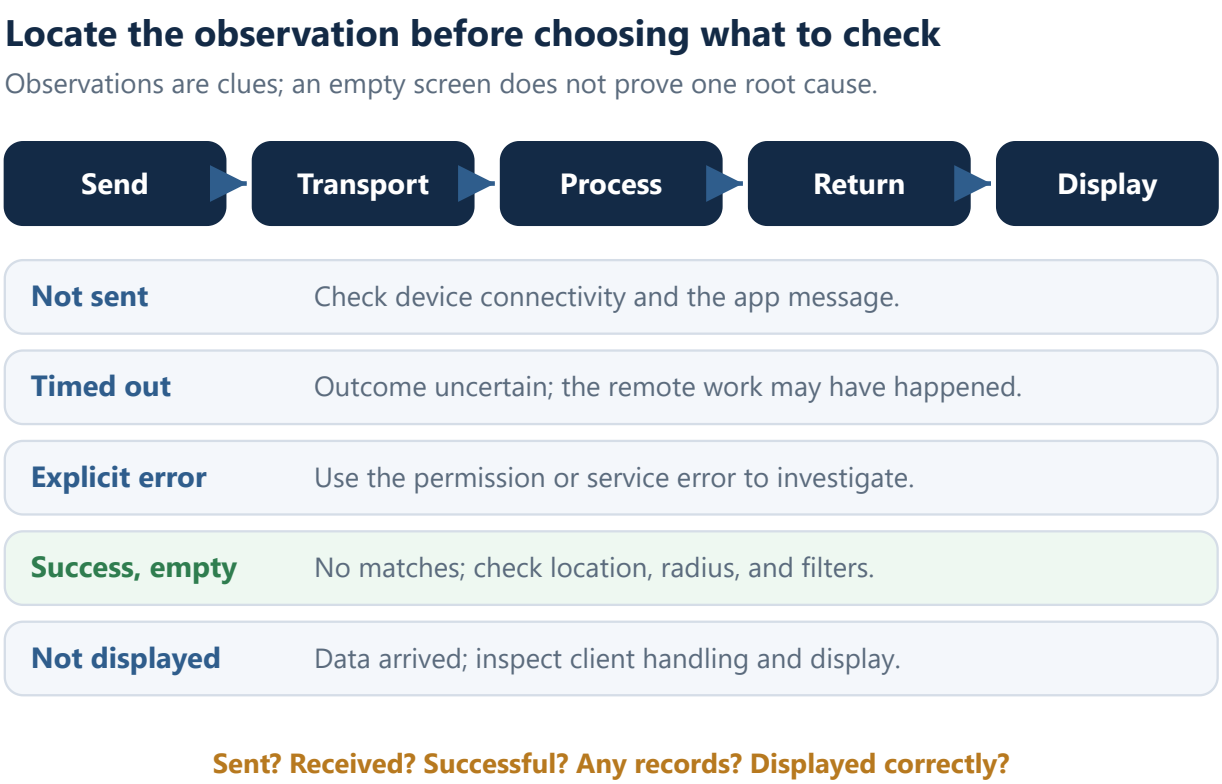


Figure 3-3. Check sending, transport, processing, return, and display. An empty list can be a successful result; a timeout leaves the outcome uncertain.

> **A tip:** a timeout does not prove that the remote operation never happened. A search can be retried, but do not repeatedly submit orders or payments just because a result did not appear. Check existing records; software also needs duplicate-prevention rules. “I did not receive the result” and “nothing happened” are different claims.

Some content may remain visible offline because the device kept earlier data, called a **cache**, or downloaded dedicated offline records. That does not prove the remote service is still reachable or the information is current. Location and connectivity are different too: some ways of determining location work offline, while a search for updated business details may still require a network.

6. Hands-on practice

Try these before reading the answers. Use the supplied fictional information; you do not need your real location, an account, or a development environment.

Exercise 1: label locations and messages

For each step in our example, write “local phone” or “remote.” Identify the two steps that send the request and response.

1. Collect “convenience store,” location A, 1,000 meters, and “open now.”
2. Send those conditions to the business-search service.
3. Check the radius, read business records, filter, and sort.
4. Send the success indicator and store list back to the phone.
5. Draw a list from the returned store records.

Then explain: why does step 4 usually not need to send the entire business database?

Exercise 2: finish an API contract

Ask a classmate to act as the search service. Use these fictional records: Sunrise is 300 meters from location A and open; Corner is 800 meters away and open; Bridge is 600 meters away and closed. Distances are supplied; no coordinate calculation is needed.

Complete the table, then walk through “location A, 1,000 meters, open only.”

What must be specified	Your agreement
Required request inputs	
Valid radius and response to an invalid value	
Fields in each successful store record	
Result ordering	
How an empty result differs from failure	

What should happen with a radius of 100 meters? What about -1 meter? Why should they not produce the same error message?

Exercise 3: draw a group-order tool across devices

Return to Lesson 1's community group-order tool. Members sign up on their phones; the organizer views totals on a computer.

Draw **a member client, the server and its saved sign-up records, and the organizer client**. Label at least these four messages with direction and contents:

1. A member submits a sign-up request.
2. The server returns the submission outcome.
3. The organizer requests a summary.
4. The server returns the summary.

Write a rule that rejects sign-ups after the deadline. Then answer: if the member sees “submitted successfully,” does that alone prove the organizer will see correct totals? What else would you check?

Optional: observe a browser request

Open a public page on a computer, open the browser's developer tools, select **Network**, and reload. Select one request and find its URL, status, and response contents. Tool locations vary by browser; finding these three items is enough.

A reload may trigger multiple requests for the page, images, and other resources. You do not need to understand every entry. Do not modify requests or upload personal information. If developer tools are inconvenient, explain Figure 3-2 aloud as a request-and-response simulation instead.

7. Suggested answers and reasoning

Exercise 1

Steps 1, 2, and 5 are local. Steps 3 and 4 are remote. Step 2 sends the request; step 4 sends the response.

The service can return just the records matching this query. Sending the whole database would include much that this search does not need. Downloading an offline database is another possible design, but it is not the design used in this example.

Exercise 2

Contract item	One suitable answer
Request	Find stores; location A; category convenience store; radius 1,000 meters; open only
Range and error	Accept 100 to 5,000 meters; return “invalid radius” for other values without performing the normal search
Each record	Store name, distance, and opening status
Ordering	Nearest first
Successful empty result	Success indicator plus empty store list; display “no stores match these conditions”

At 1,000 meters, return Sunrise at 300 meters, then Corner at 800 meters. Bridge is excluded because it is closed. At 100 meters, the query is valid but has no matches: return success and an empty list. At -1 meter, return an input error.

One means “we checked and found no matches.” The other means “the conditions were invalid.” The display should help the user change the appropriate input. Different field names are fine if both sides follow the same agreement.

Exercise 3

Message	Direction	Minimum contents
Sign-up request	Member client to server	Group-order batch, member, items, quantities
Submission outcome	Server to member client	Whether saving succeeded; record identifier or failure reason
Summary request	Organizer client to server	Requested batch; access credentials if required by the actual system
Summary response	Server to organizer client	Totals per item and any required sign-up details

The server saves sign-ups, and summary processing reads records for the same batch. It should check the deadline using its own time and explicitly reject late submissions. Disabling a button on the phone alone does not enforce that rule.

“Submitted successfully” is only an interface message. Check that the record was saved, belongs to the right batch, is included in the summary, and contributes correctly to the totals. Use two fictional members: one orders two portions of greens, the other three. The organizer should see five. A late sign-up should be rejected without changing that total.

Your drawing can look different. It succeeds if you can explain the four messages, where records are saved, and how the summary is produced.

Checkpoint: after this lesson you should be able to...

- Draw local and remote responsibilities in the map example and explain that placement is a design choice.
- Identify the client and server and describe the capability being used.
- Follow a request and response, naming what each arrow carries.
- Specify an API's operation, inputs, successful result, and errors in your own words.
- Distinguish no response, explicit failure, successful empty results, and display problems after data arrives.

Test yourself: without looking back, explain in two minutes how tapping a button on your phone can ask a remote program to look up records. Being able to answer follow-up questions along your diagram matters more than reciting the vocabulary.

Next lesson

Software divides work by responsibility, and parts on different devices can cooperate through requests and responses. Later, the remote capability might be a model or a search tool. A failed call does not necessarily mean the model itself is at fault.

Lesson 4, *From Design to Running*, connects the diagrams and explanations from Lessons 1 to 3 to a small example that actually runs. You will see how a design becomes files and a program, how input becomes output, and how to check results against the original requirements. Lesson 5 then examines files, environments, dependencies, and operating systems more systematically.

Further reading (optional)

These official references support the terminology and offer more detail. They are not prerequisites for the exercises.

- MDN: Client-server overview: the basic request-and-response exchange.
- MDN: API: interfaces can expose local capabilities as well as remote ones.
- MDN: How the web works: addresses, DNS, and a basic access path.
- MDN: HTTPS and HTTP response status codes: transport protection and result status.